

Function expression

25 August 2026 18:54

In JavaScript a function is not a "Magical language structure", but a special kind of value.

The syntax that we used before is called Function Declaration:

```
function sayHi(){
  console.log(`Hello!`);
}
```

There is another syntax for creating a function that is called a function expression. It allow us to create a new function in the middle of any expression.

For example:

```
let sayHi = function(){
  console.log(`Hello!`);
};
```

Here we can see a variable *sayHi*, getting a value. The new function created as `function(){console.log(`Hello`); }`;

As the function creation happens in the context of the assignment expression (to the right side of `=`) this is a function expression.

Please note there's no name after the function keyword. Omitting a name is allowed for function expression.

Here we immediately assign it to the variable, so the meaning of these code samples is the same: "create a function and put it into the variable *sayHi*".

In more advanced situations that we will come across later, a function may be created and immediately called or scheduled for a later execution, not stored anywhere thus remaining anonymous.

Function is a value

Let's reiterate. No matter how the function is created, a function is a value. Both examples above store a function in the, *sayHi* variable.

We can even print out that value using `console.log()`:

```
function sayHi(){
  console.log(`hello!`);
}

console.log(sayHi); //method as value
sayHi(); //method call
```

Output:

```
[Function: sayHi]
hello!
```

Please note that the line before last line does not run the function because there are no parentheses after `sayHi`. There are programming languages where any mention of a function name causes its execution, but Javascript is not like that.

In Javascript, a function is a value, so we can deal with it as a value. The code above shows it's String representation which is the source code.

Is a special value in the sense that we can call it like `sayHi()`.

But it's still a value, so we can work with it like with other kinds of values.

We can copy a function to another variable.

```
function sayHi() { //1. create
  console.log(`hello!`);
}
console.log(sayHi);
let func1 = sayHi; // 2. copy
func1(); //Hello 3 run the copy (it works)
sayHi(); //hello // this still works too
```

Output:

```
[Function: sayHi]
hello!
hello!
```

Here's what happens above in detail:

1. The function declaration (1) creates the function and puts it into the variable name `sayHi`.
2. Line (to) copies it into the variable `func`. Please note again: There are no parentheses after `sayHi`. If there were, then `func = sayHi()` would write the result of the call `sayHi()` into `func`, not the function `sayHi` itself.
3. Now the function can be called as both `sayHi()` and `func()`.

We could also have used a function expression to declare, `sayHi` in the first line:

```
let sayHi = function(){ //1. create
  console.log();
};

let func = sayHi; //2. copy
//...
```

Everything would work the same

Why is there a semicolon at the end?

You might wonder why do function expression have a ; at the end but function declaration do not:

```
function sayHi(){
  //...
}

let sayHi = function(){
  //...
};
```

The answer is simple: A function expression is created here as function(...) {...} inside the assignment statement: let sayHi = ...;. The semicolon ; is recommended at the end of the statement, it's not the part of the function syntax.

The semicolon ; would be there for a simple assignment such as let sayHi= 5;; and it's also there for a function assignment.

Callback functions

Let's look at the more examples of passing functions as values and using function expression. We will write a function ask(question, yes no) With three parameters:

question

Text of the question

yes

Function to run if the answer is "yes".

no

Function to run if the answer is "no".

The function should ask the question and depending on the users answers call yes() or no():

```
const prompt = require('prompt-sync')();
function ask(question, yes, no) {
  let q = prompt(`${question}`)
  if (q === `yes`) {
    yes();
  }
  else {
    no();
  }
}
function showOk() {
  console.log(`You agreed`);
}
function showCancel(){
  console.log(`You canceled the execution`);
}
```

yes = showOk

no = showCancel

This is due to
Function Expression

```
}  
function showCancel(){  
  console.log(`You canceled the execution.`)  
}  
ask(`Do you agree?`, showOk, showCancel)
```

Output:

- 1) Do you agree?yes
You agreed
- 2) Do you agree?no
You canceled the execution.

In practice, such functions are quite useful. The major difference between a real-life ask and the example above is that real-life functions are more complex ways to interact with the user than a simple confirm. In the browser such function usually draw a nice - looking question window. But that's another story.

The arguments showOk and showCancel of ask are called **callback** function or just **callbacks**.

The idea is that we pass a function and expect it to be called back later if necessary. In our case, **showOK** becomes the callback for **Yes** answer and **showCancel** for **No** answer.